CS3230 : Design and Analysis of Algorithms (Spring 2014)

Homework Set #3

[Lectures 6 to 11]

OUT: Thu, 27-Mar-2014
Due: Fri, 11-Apr-2014, before 12:00 noon @ dropbox outside Prof Leong's office

Q1. (10 marks) (Modified from Problem 17.4-4 of [CLRS]**) More elegant potential function**

Suppose that instead of contracting a table by halving its size when its load factor drops below ¼, we contract it by multiplying its size by 2/3 when its load factor drops below 1/3. Using the potential function

$\Phi(T) = |\, 2num[T] - size[T] \,|$,

show that the amortized cost of a TABLE-INSERT and TABLE-DELETE that uses this strategy is bounded above by a constant.

(*Hint*: When dealing with this $\Phi(T)$, consider the case when $2num[T] - size[T] \geq 0$ separately from the case when $2num[T] - size[T] \leq 0$.)

(a) **(5 marks)** Analyze, fully, the *amortized cost* of TABLE-INSERT using potential method.
(*Hint:* Further separate into case when the table will *expand* and when it will not.)

(b) **(5 marks)** Analyze, fully, the *amortized cost* of TABLE-DELETE using potential method.
(*Hint:* Further separate into case when the table will *contract* and when it will not.)

SOLUTION:

(a) First, analyze the INSERT operation:

Suppose $num[T] = size[T]$
then INSERT will trigger an EXPAND and $(2num[T] - size[T]) > 0$)
and so actual cost $c_i = 1 + size[T]$ (expand, move-old-elts, insert-new-elt);
Then $size[Tnew] = 2size[T]$. (since, we EXPAND)

Also $\Phi(T) = |\, 2num[T] - size[T] \,| = (2num[T] - size[T])$
and so $\Phi(Tnew) = |\, 2(num[T]+1) - size[Tnew] \,| = (2num[T] + 2 - 2size[T])$
Change in potential $= 2 - size[T]$
Amortized cost $c_i' = (1 + size[T]) + (2 - size[T]) = 3$ ($1 to insert, $2 saved)
(use the *saved-potential* for move old-elt)

Suppose $size[T]/2 \leq num[T] < size[T]$
then INSERT will NOT trigger EXPAND and $(2num[T] - size[T]) \geq 0$)
and so actual cost $c_i = 1$ (just insert);
Then $size[Tnew] = size[T]$. (since no EXPAND)

Also $\Phi(T) = |\, 2num[T] - size[T] \,| = (2num[T] - size[T])$
and so $\Phi(Tnew) = |\, 2(num[T]+1) - size[T] \,| = (2num[T] + 2 - size[T])$
Change in potential $= 2$
Amortized cost $c_i' = 1 + 2 = 3$ ($1 to insert, $2 saved per node)

Suppose $size[T]/3 \le num[T] < size[T]/2$
then INSERT will NOT trigger EXPAND and $(2num[T] - size[T]) \le 0)$
and so actual cost $c_i = 1$ (just insert);
Then $size[Tnew] = size[T]$. (since no EXPAND)

Also $\Phi(T) = | 2num[T] - size[T] | = (size[T] - 2num[T])$
and so $\Phi(Tnew) = | size[T] - 2(num[T]+1) | = (size[T] - 2num[T] - 2)$
Change in potential = –2
Amortized cost $c_i' = 1 - 2 = -1$ (Note: this means, we "throw away" $1 credit)

(b) Now, analyze the DELETE operation:

Suppose $size[T]/2 < num[T] \le size[T]$:
then DELETE will NOT trigger CONTRACT and $(2num[T] - size[T]) \ge 0)$
and so actual cost $c_i = 1$ (just remove the item);
and $size[Tnew] = size[T]$. (since no CONTRACT)

Also, $\Phi(T) = | 2num[T] - size[T] | = (2num[T] - size[T])$
and so $\Phi(Tnew) = | 2(num[T]-1) - size[Tnew] | = (2num[T] - 2 - size[T])$
Change in potential = –2
Amortized cost $c_i' = 1 + -2 = -1$ (Note: this means, we "throw away" $1 credit)

Suppose $size[T]/3 < num[T] \le size[T]/2$
then DELETE will NOT trigger CONTRACT and $(2num[T] - size[T]) \le 0)$
and so actual cost $c_i = 1$ (just remove the item);
Then $size[Tnew] = size[T]$. (since no CONTRACT)

Also $\Phi(T) = | 2num[T] - size[T] | = (size[T] - 2num[T])$
and so $\Phi(Tnew) = | size[T] - 2(num[T]-1) | = (size[T] - 2num[T] + 2)$
Change in potential = 2
Amortized cost $c_i' = 1 + 2 = 3$ ($1 to delete, $2 saved per node)

Suppose $num[T] = size[T]/3 + 1$;
then DELETE will trigger CONTRACT and $(2num[T] - size[T]) < 0)$
and so actual cost $c_i = 1 + size[T]/3$ (delete, contract, move-old-elts);
Then $size[Tnew] = 2size[T]/3$. (since we CONTRACT)

Also $\Phi(T) = | 2num[T] - size[T] | = (size[T] - 2num[T])$
and so $\Phi(Tnew) = | size[Tnew] - 2(num[T]-1) | = (2size[T]/3 - 2num[T] + 2)$
Change in potential = $2 - size[T]/3$
Amortized cost $c_i' = (1 + size[T]/3) + (2 - size[T]/3) = 3$ ($1 to delete, $2 saved)
(use the *saved-potential* for move old-elt)

Remarks from Prof Leong Hon Wai (Q1 grader): More students got it. One common (small) error was not to consider the sign of the expression inside the | | sign of the potential function This expression can be positive or NEGATIVE and so it is best to treat them separately.

I am also disappointed that some students obviously "borrowed" solution(s) from the web WITHOUT ACKNOWLEDGE IT. This is PLAGIARISM. Very neat solution, despite the rather tedious calculations and same exact mistakes made in very neat solutions. And also all the cases considered in exactly the same order. I decided to be lenient and let it go. BUT, if you are one of them, take it as a WARNING.

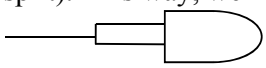
Q2. (10 marks) Lecture 07+08: NP-Completeness

a. **(2 marks) CIRCUIT-SAT $\leq_p$ SAT**
Give a small circuit with just 7 gates (any combination of AND, OR, or NOT gates), but if reduced to SAT instance wrongly (using the non polynomial reduction shown in Lecture 08), we will have an exponential number of variables. Note that there can be several valid answers, therefore to help grading, you <u>need to explain</u> why the circuit that you construct will generate exponential number of variables if reduced wrongly (0 marks without such explanation).

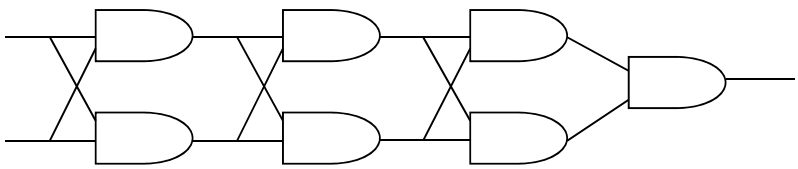
Finally, show the same reduction using the correct polynomial time reduction algorithm as shown in Lecture 08.

Solution:

The most extreme solution is to copy this gate 7 times (In CIRCUIT-SAT, wire is allowed to split). This way, we have $\phi = (x_7 \wedge x_7) = (x_6 \wedge x_6) \wedge (x_6 \wedge x_6) = \ldots$ double every time…



Other alternative solutions exist, like the one below:



$\phi_2 = x_9 = (x_7 \wedge x_8) = ((x_5 \wedge x_6) \wedge (x_5 \wedge x_6)) = (((x_3 \wedge x_4) \wedge (x_3 \wedge x_4)) \wedge ((x_3 \wedge x_4) \wedge (x_3 \wedge x_4))) =$
$((((x_1 \wedge x_2) \wedge (x_1 \wedge x_2)) \wedge ((x_1 \wedge x_2) \wedge (x_1 \wedge x_2))) \wedge ((x_1 \wedge x_2) \wedge (x_1 \wedge x_2)) \wedge ((x_1 \wedge x_2) \wedge (x_1 \wedge x_2))))$
If we add one more layer of 2 gates, we will create 16*2 = 32 more variables, and so on.
This is clearly exponential.

The correct reduction:
$\phi_3 = x_9 \wedge (x_9 \leftrightarrow (x_7 \wedge x_8)) \wedge (x_7 \leftrightarrow (x_5 \wedge x_6)) \wedge (x_8 \leftrightarrow (x_5 \wedge x_6)) \wedge (x_5 \leftrightarrow (x_3 \wedge x_4)) \wedge (x_6 \leftrightarrow (x_3 \wedge x_4)) \wedge$
$(x_3 \leftrightarrow (x_1 \wedge x_2)) \wedge (x_4 \leftrightarrow (x_1 \wedge x_2))$
This formula ϕ_3, although looks longer than ϕ_2 and has #gates+1 = 7+1 = 8 clauses, grow linearly. If we add one more layer of 2 gates, we will just add 2 more clauses.

b. **(2 marks) SAT $\leq_p$ 3-CNF-SAT**
Suppose that we have the following short SAT instance $\phi = y_1 \wedge (y_1 \leftrightarrow (\neg x_1 \wedge x_2))$.
Reduce this short SAT instance ϕ into a 3-CNF-SAT instance ϕ' using the second of the two steps reduction shown in class (notice that the first step is already implied in ϕ).

Solution:

The truth table for $(y_1 \leftrightarrow (\neg x_1 \wedge x_2))$ with extra columns to help understanding is as follows:

y_1	x_1	$\neg x_1$	x_2	$(\neg x_1 \wedge x_2)$	$(y_1 \leftrightarrow (\neg x_1 \wedge x_2))$
1	1	0	1	0	0
1	1	0	0	0	0
1	0	1	1	1	1
1	0	1	0	0	0
0	1	0	1	0	1
0	1	0	0	0	1
0	0	1	1	1	0
0	0	1	0	0	1

The 3-DNF is thus $(y_1 \wedge x_1 \wedge x_2) \vee (y_1 \wedge x_1 \wedge \neg x_2) \vee (y_1 \wedge \neg x_1 \wedge \neg x_2) \vee (\neg y_1 \wedge \neg x_1 \wedge x_2)$.
The 3-CNF is thus $(\neg y_1 \vee \neg x_1 \vee \neg x_2) \wedge (\neg y_1 \vee \neg x_1 \vee x_2) \wedge (\neg y_1 \vee x_1 \vee x_2) \wedge (y_1 \vee x_1 \vee \neg x_2)$.

As the first clause y_1 is a 1-literal clause, we need to transform it into:
$(y_1 \vee p \vee q) \wedge (y_1 \vee p \vee \neg q) \wedge (y_1 \vee \neg p \vee q) \wedge (y_1 \vee \neg p \vee \neg q)$.

We combine everything together to have the following 3-CNF instance ϕ':
$(y_1 \vee p \vee q) \wedge (y_1 \vee p \vee \neg q) \wedge (y_1 \vee \neg p \vee q) \wedge (y_1 \vee \neg p \vee \neg q) \wedge$
$(\neg y_1 \vee \neg x_1 \vee \neg x_2) \wedge (\neg y_1 \vee \neg x_1 \vee x_2) \wedge (\neg y_1 \vee x_1 \vee x_2) \wedge (y_1 \vee x_1 \vee \neg x_2)$.

c. (6 marks) Prove that KNAPSACK is NP-Complete.

KNAPSACK problem: Consider a set of **n** items where the i-th item is of weight w_i and of value v_i. Suppose the maximum load of your car is W and $w_i \leq W$ for all i. We aim to find a subset of items $S \subseteq \{1, 2, \ldots, n\}$ such that the value $\sum_{i \in S} v_i$ is maximized while $\sum_{i \in S} w_i \leq W$. The decision version of the problem asks if its total value $\sum_{i \in S} v_i \geq K$ for a value K.

(c.i) (2 mark) Show that KNAPSACK problem is in NP.

Solution:
The certificate is the set of items $S \subseteq \{1, 2, \ldots, n\}$ that you selected. Then, in polynomial time, we can check if $\sum_{i \in S} w_i \leq W$ and $\sum_{i \in S} v_i \geq K$. Hence, the Knapsack problem is clearly in NP. Optional: The certificate is already a valid set so we do not need to specifically check for uniqueness and validity of the set anymore.

(c. ii) (4 marks) Prove that there exists a polynomial-time reduction from the SUBSET-SUM problem to the KNAPSACK problem. Describe your reduction clearly.

Definition of the SUBSET-SUM problem: Given a set A of positive integers and an integer s, the SUBSET-SUM problem decides if there exists a subset B of A whose sum is s.

After you have proven your polynomial-time reduction, use it to give the Knapsack problem instance given this subset-sum instance A = {1, 2, 5, 9, 20} and s = 22.

Solution:

The input of the subset-sum problem is (A, s). We transform it to the Knapsack problem as follows: For every A_i in A, set $w_i = v_i = A_i$. We also set W = K = s. This is O(n), polynomial.

Claim: SUBSET-SUM(A, s) is true iff KNAPSACK(w_i, v_i, W, K) is true.

(→) Let B be the subset of A such that $\sum_{i \in B} A_i = s$.
Then, $\sum_{i \in B} w_i = \sum_{i \in B} A_i = s \leq$ W. $\sum_{i \in B} v_i = \sum_{i \in B} A_i = s \geq K$.
Hence, KNAPSACK(wi, vi, W, K) is true.

(←) Let B be the subset such that $\sum_{i \in B} w_i \leq$ W and $\sum_{i \in B} v_i \geq K$.
Since $w_i = v_i = A_i$ and W = K, we have $\sum_{i \in B} A_i = s$.
Hence, SUBSET-SUM(A, s) is true.

For the given example, we set (w1, w2, w3, w4, w5) = (v1, v2, v3, v4, v5) = (1, 2, 5, 9, 20).
W = K = 22.

Remarks from Steven (Q2 grader):

Part a is one of the hardest and "original" open-ended question which is hard to grade. Students cannot Google around for solution and there are lots of variety of solutions here ☺. Eventually I settle for the best (one line that split to two with 7 and/or gates) and second best answer for 2 marks, and any non-optimal answer is given 1 mark only.

For part b, some students make minor mistakes in the reduction (only 4 CNF clauses are generated). Some students also forget to expand y1 (single literal) into 4 clauses with exactly 3 distinct literals. I accept answers that eliminate duplicate clauses and those who leave duplicate clauses as they are.

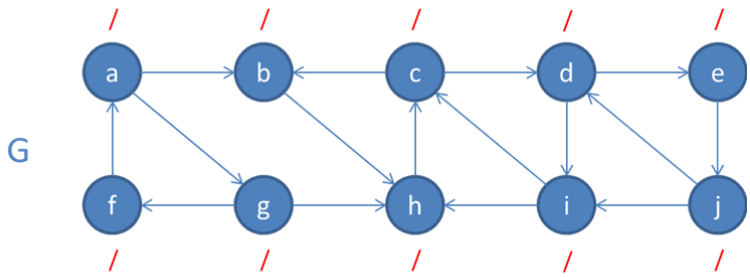
For part c, the common mistake is in verifying KNAPSACK is in NP. Look at the wording carefully. In the decision version, you are already given the target K to achieve.

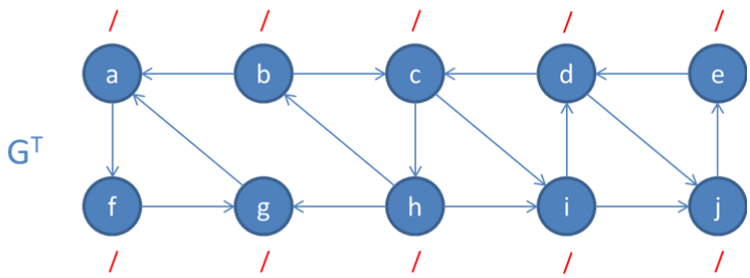
Q3. (10 marks) Lecture 09: Graph Algorithms

Consider the following directed graph G and its transpose G^T. Find all strongly connected components of G by executing Kosaraju's Strongly Connected Component algorithm (please do NOT use another SCC finding algorithm to simplify our grading process). Please answer the following questions.

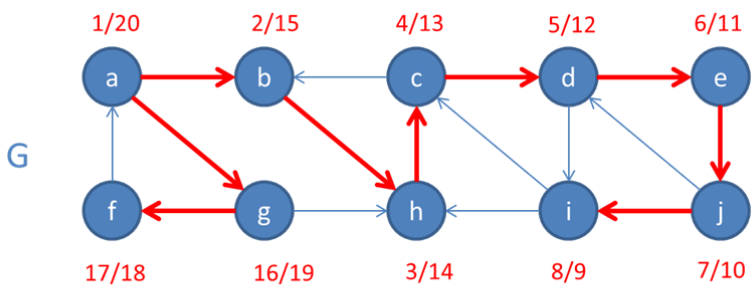
a. (4 marks) DFS traversal

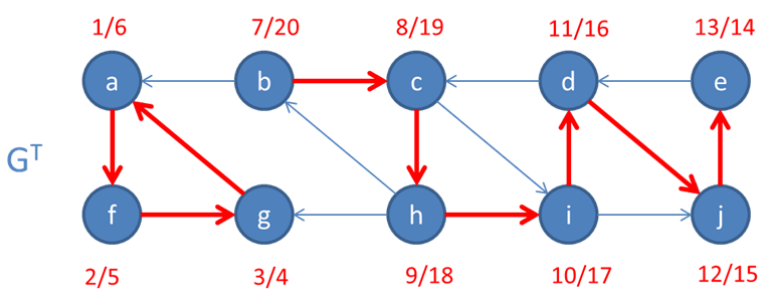
For each vertex in both G and G^T, please mark the discovery time and the finishing time when we execute the Kosaraju's Strongly Connected Component algorithm. Please also draw the edges of the DFS tree in both G and G^T. (When you perform DFS in G, please assume you will visit the vertices in alphabetical order.)





Solution:





Printed -

b. (3 marks) The SCCs

From the result in (3.a), please write down the Strongly Connected Components.

Solution:

There are two strongly connected components.
They are {a, f, g} and {b, c, d, e, h, i, j}.

c. (3 marks) types of DFS tree edges

In G, write down the edge types of (g, h), (a, b), (c, b), (d, i) and (b, h). (Note that there are 4 edge types: tree edge, back edge, forward edge and cross edge.)

Solution:

(g, h): _____cross edge_____
(a, b): _____tree edge_____
(c, b): _____back edge_____
(d, i): _____forward edge_____
(b, h): _____tree edge_____

Remarks from Jing Wei (Acer) (Q3 grader): Question 3 is very standard and very few people (less than 10) got it wrong. When they get it wrong, the mistake is usually involve the second step of Kosaraju's SCC finding algorithm.

Q4. (10 marks) Lecture 10+11: Dynamic Programming and Greedy Algorithms

SUBSET-SUM Problem. Let A[1..n] be a set of positive integers (sorted in increasing order), and s is a positive integer. Is there any subset B of A such that the sum of the elements in B equals to s? For example, A = (1, 2, 5, 9, 20), s = 22. Then, B = (2, 20).

(a). (6 marks) Super-increasing sequence.
A super-increasing sequence A is a sequence such that the next term of the sequence is greater than the sum of all preceding terms. In other words,
$A_{k+1} > \sum_{j=1}^{k} A_j$, where A_j is the jth term of A.
For example, A = (1, 2, 5, 9, 20) is a super-increasing sequence, because 2>1; 5>1+2; 9>1+2+5; and 20>1+2+5+9.

(a.1). (1 mark)
For A = (1, 2, 5, 9, 20), find the corresponding set B for the integer s = 32, s = 25, s = 23.

Solution:

s = 32: _____ {1, 2, 9, 20}
s = 25: _____ {5, 20}
s = 23: _____ {1, 2, 20}

(a.2). (2 marks) Design!
Please propose a greedy algorithm which solves the SUBSET-SUM problem when A is super-increasing.

Solution:

A greedy algorithm that always chose the largest possible value first

```
B = Φ;
set A0 = 0;
k = n;
while (s > 0 and k > 0) do
     while (Ak > s and k > 0) do
          k = k - 1; // find the biggest integer Ak ≤ s;
     B = B ∪ {Ak};
     s = s - Ak;
if (s = 0) report B else report "fail!";
```

PS: There are OTHER valid ways to write the short iterative solution.

(a.3). (1 mark) Analyze!
What is the running time of your algorithm?

Solution:

Although the code has two while loops, the running time is O(n), where n is the size of A because k starts from n, decreases by 1 in the inner while loop and never increases again. We can only decrease k n times before this algorithm stops.

(a.4.) (2 marks) Prove!
Prove that your proposed greedy algorithm is correct.

Solution:

If there is no subset sum or the greedy algorithm finds s, the output of the greedy algorithm is obviously correct.

The key part is in the super-increasing property of A.

Let $A_{biggest}$ be the biggest number in A that is less than or equal to s. We have to take $A_{biggest}$ in the optimal solution because if we don't, we will not be able to get s as the sum of ALL other numbers before $A_{biggest}$ in A is strictly less than $A_{biggest}$.

PS: Other valid arguments are accepted.

b. (4 marks) Dynamic Programming.
Professor Perfect thinks that dynamic programming can be used to solve the problem when A is not super-increasing.
Please provide a recurrence relation for the dynamic programming algorithm which solves the SUBSET-SUM for any A. Justify the correctness of the recurrence relation proposed **(2 marks)** and show that this recurrence relation can generate overlapping sub-problems **(2 marks).**

Solution:

Let V(i, t) equals 1 if there exists a subset of {A1, A2, ..., Ai} whose sum is s.

Base case: V(0, s) = 0 and V(i, 0) = 1.

Recursive case: V(i, s) = 1 if V(i-1, s-Ai) = 1 (take this item Ai) or V(i-1, s) = 1 (ignore this item Ai).

This recurrence relation captures all possible actions that we can do with each item, therefore it is correct.

This recurrence relation generates overlapping subproblems, for example, if all items in A have similar value, then for example, taking the first item and ignoring the rest is the same as taking the second item and ignoring the rest. This is one of the many potential overlapping subproblems in this DP formulation.

Note: This DP is a pseudo polynomial algorithm.

Remarks from Zhou Yiming (Q4 grader): Two common mistakes are:
1. Part 4(a) 2, students forgot to verify S = 0 at the end, they just give the set as the final answer. (1 mark deducted)
2. Part 4(b), students forgot to answer overlapping(-2) or their explanation is not good enough(-1).